

# TEORÍA DE LA COMPUTACIÓN

## Tarea: análisis léxico del lenguaje Prolog

*Investigación, análisis, diseño, implementación y validación*

### 1. Objetivo de la tarea

Analizar, diseñar e implementar un analizador léxico para un subconjunto definido del lenguaje Prolog, aplicando alfabetos, lenguajes regulares, expresiones regulares y autómatas finitos. La tarea busca integrar los fundamentos formales de la Teoría de la Computación con una implementación capaz de reconocer los elementos léxicos propios de Prolog, administrar lexemas y detectar errores léxicos de manera verificable.

### 2. Contexto del problema

Se requiere desarrollar un analizador para archivos fuente de Prolog. Este recibirá un programa o consulta, recorrerá sus caracteres de izquierda a derecha y producirá una secuencia ordenada de tokens. Deberá distinguir átomos, variables, números, cadenas, operadores y delimitadores; ignorar correctamente espacios y comentarios; y reportar símbolos o lexemas no válidos indicando su ubicación. Para acotar el trabajo se utilizará el subconjunto de Prolog especificado en este enunciado, inspirado en la notación habitual de ISO Prolog y SWI-Prolog.

**Alcance obligatorio:** la tarea se centra únicamente en análisis léxico. No se solicita analizar la estructura sintáctica de instrucciones, construir árboles sintácticos ni validar el orden gramatical de los tokens.

### 3. Categorías léxicas de Prolog

El analizador deberá reconocer, al menos, las siguientes categorías del subconjunto de Prolog que deberá investigar primero, por ejemplo:

- Átomos no entrecomillados, iniciados con letra minúscula y continuados por letras, dígitos o guion bajo: padre, persona\_1.
- Átomos entre comillas simples, incluyendo espacios o caracteres especiales: 'Juan Pérez', ':-'.
- Variables, iniciadas con letra mayúscula o guion bajo: X, Persona, \_Temporal; y la variable anónima \_.
- Números enteros y reales, incluyendo signo únicamente cuando el criterio definido por el grupo permita distinguirlo del operador aritmético.

- Cadenas delimitadas por comillas dobles, considerando una convención documentada para secuencias de escape.
- Operadores de cláusulas y consultas: `:-`, `?-` y, opcionalmente, `-->`.
- Operadores de unificación y comparación: `=`, `\=`, `==`, `\==`, `=..`, `<`, `=<`, `>` y `>=`.
- Operadores aritméticos: `+`, `-`, `*`, `/`, `//`, `**`; y átomos operadores como `is` y `mod`.
- Operadores de control: `\+`, `!` y punto y coma; además de coma como separador/conjunción.
- Delimitadores: `(`, `)`, `[`, `]`, `{`, `}`, barra vertical y punto final de cláusula.
- Comentarios de línea iniciados con `%` y comentarios de bloque `/* ... */`.
- Espacios en blanco, tabulaciones y saltos de línea, que deberán ignorarse conservando la posición.

#### 4. Trabajo solicitado

1. **Especificación del subconjunto:** Definir formalmente qué formas léxicas de Prolog serán admitidas. Para cada categoría se indicará nombre del token, patrón, ejemplos válidos, ejemplos inválidos y atributo asociado cuando corresponda.
2. **Expresiones regulares:** Construir una expresión regular para cada categoría léxica. Deben explicitarse las convenciones utilizadas y resolverse las posibles superposiciones entre patrones.
3. **Modelado con autómatas:** Construir AFN o AFD para las categorías principales y presentar un AFD integrado o una estrategia equivalente. Deben identificarse estado inicial, estados de aceptación y token reconocido.
4. **Determinización y minimización:** Mostrar, para un subconjunto representativo de tokens, la conversión de AFN a AFD mediante construcción de subconjuntos y aplicar minimización al AFD resultante, justificando estados equivalentes.
5. **Implementación:** Programar el analizador en Python, Java o C++. Por cada token deberá generar tipo, lexema, línea y columna; opcionalmente podrá incluir un atributo o referencia a tabla de símbolos.
6. **Tabla de lexemas:** Registrar al menos átomos, variables y literales sin duplicados innecesarios, indicando el atributo o índice conservado por el token. No se solicita resolver ámbitos, unificación ni otro análisis semántico.
7. **Errores léxicos:** Detectar caracteres no admitidos, átomos o cadenas sin cierre, comentarios de bloque sin cierre y números mal formados. El mensaje deberá incluir ubicación y fragmento problemático, procurando continuar el análisis.
8. **Validación:** Diseñar y ejecutar un corpus de pruebas que cubra todas las categorías, los límites entre tokens, las prioridades y los errores solicitados.

#### 5. Ejemplo de entrada y salida esperada (esto es solo un ejemplo)

```
% Hechos
padre(juan, ana).
padre(juan, pedro).
```

```
% Regla
abuelo(X, Z) :-
    padre(X, Y),
    padre(Y, Z).

?- abuelo(juan, Quien).
```

Fragmento ilustrativo de la salida:

```
<ATOMO, 'padre', 2, 1>
<PARENTESIS_IZQ, '(', 2, 6>
<ATOMO, 'juan', 2, 7>
<COMA, ',', 2, 11>
<ATOMO, 'ana', 2, 13>
<PARENTESIS_DER, ')', 2, 16>
<PUNTO, '.', 2, 17>
```

## 6. Casos de prueba mínimos

El repositorio deberá incluir un corpus reproducible con, al menos:

- Al menos 20 pruebas válidas, cubriendo todas las categorías léxicas de Prolog definidas.
- Al menos 8 pruebas inválidas, incluyendo átomos o cadenas sin cierre, comentarios sin cierre, caracteres no admitidos y números mal formados.
- Casos específicos de máxima coincidencia y prioridad entre átomos, variables y operadores de uno o varios caracteres.
- 2 archivos de entrada completos: uno sin errores y otro con múltiples errores léxicos recuperables.

## 7. Entregables

- Informe técnico en PDF (**Tarea\_JefeGrupo\_nombreApellido.pdf**), con la especificación de tokens, expresiones regulares, autómatas, proceso de determinización/minimización, decisiones de prioridad, diseño del lexer, pruebas, resultados y conclusiones.
- Repositorio Git con código fuente, instrucciones de ejecución, dependencias, corpus de pruebas e historial de contribuciones. El enlace deberá incorporarse en el anexo del informe y compartirse con el profesor.

## 8. Estructura sugerida del informe

- Portada y resumen ejecutivo.
- Introducción, objetivo y alcance.
- Especificación del lenguaje y de los tokens.
- Subconjunto de Prolog, catálogo de tokens y expresiones regulares.

- Construcción de AFN/AFD, determinización y minimización.
- Diseño e implementación del analizador léxico.
- Prioridad, máxima coincidencia, tabla de lexemas y errores léxicos.
- Plan de pruebas, resultados y discusión.
- Conclusiones, referencias y anexos.

## 9. Criterios de evaluación

| Dimensión                              | Aspectos principales                                          | Puntos |
|----------------------------------------|---------------------------------------------------------------|--------|
| Especificación y expresiones regulares | Corrección, cobertura y claridad formal.                      | 20     |
| Modelado con autómatas                 | AFN/AFD, determinización, minimización y justificación.       | 25     |
| Implementación del lexer               | Corrección, prioridades, posiciones, modularidad y ejecución. | 25     |
| Pruebas y errores léxicos              | Cobertura, reproducibilidad y calidad de diagnósticos.        | 15     |
| Informe, repositorio y presentación    | Argumentación, evidencia, trazabilidad y comunicación.        | 15     |

## 10. Condiciones y recomendaciones

- Todos los integrantes deben comprender la solución completa y registrar contribuciones en Git.
- Las expresiones regulares y los autómatas deben ser coherentes con el código entregado; no basta presentar definiciones teóricas desvinculadas de la implementación.
- Las fuentes consultadas deben citarse con un formato académico consistente.
- Se valorará la automatización de pruebas y una visualización clara de los autómatas, sin que ello reemplace los requisitos obligatorios.
- El plagio, la copia de repositorios o la imposibilidad de explicar el código se regirán por la normativa académica vigente.

### **Ver en plataforma educa:**

- ✓ Carpeta: Formatos
- ✓ Plazo